

**Pembuatan Aplikasi Chat Web dengan Kriptografi Kunci-Simetri dan
Kunci-Publik
II4021 - Kriptografi**



Penulis:

Ferro Arka Berlian	18223027
Atharizza Muhammad Athaya	18223079
Muhammad Omar Berliansyah	18223055

**SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG**

2026

DAFTAR ISI

DAFTAR ISI.....	1
PERNYATAAN AKADEMIK.....	2
BAB I	
DASAR TEORI.....	3
1.1. JSON Web Token (JWT).....	3
1.2. Elliptic Curve Diffie-Hellman (ECDH).....	3
1.3. Advanced Encryption Standard (AES).....	4
1.4. Password-based MAC Key Derivation Function (PBKDF2).....	5
1.5. Hash-based MAC Key Derivation Function (HKDF).....	6
1.6. Hashing SHA-256.....	7
1.7. Message Authentication Code (MAC).....	7
1.8. Dockerization.....	8
BAB II	
PERANCANGAN DAN IMPLEMENTASI.....	9
2.1. Abstraksi dan Arsitektur Sistem.....	9
2.2. Registrasi dan Manajemen Pengguna.....	9
2.3. Autentikasi Pengguna (JWT).....	9
2.4. Pembentukan Kunci Komunikasi (Key Exchange).....	10
2.5. Enkripsi dan Dekripsi Pesan.....	11
2.6. Implementasi Bonus.....	12
BAB III	
PENGUJIAN DAN HASIL.....	13
3.1. Uji Autentikasi dan Manajemen Pengguna.....	13
3.2. Unit Testing.....	18
3.3. Enkripsi dan Dekripsi Pesan.....	21
BAB IV	
KESIMPULAN.....	22
DAFTAR PUSTAKA.....	23
LAMPIRAN.....	24

PERNYATAAN AKADEMIK

Kami menyatakan bahwa kode program yang dihasilkan bukan merupakan hasil salinan mentah (raw output) dari Generative AI, melainkan hasil pengembangan dan penulisan mandiri.



Ferro Arka Berlian



Atharizza Muhammad
Athaya



Muhammad Omar
Berliansyah

BAB I

DASAR TEORI

1.1. JSON Web Token (JWT)

JSON Web Token (JWT) merupakan teks panjang yang berfungsi sebagai *enabler* autentikasi *stateless* menggunakan rumus matematika untuk menyimpan sesi user daripada menyimpan semuanya di database. Adapun struktur JWT sendiri terbagi menjadi tiga bagian, yaitu *Header* yang berisi deskripsi token itu sendiri (seperti algoritma yang digunakan, tipe token, dan lain-lain), *Payload* yang merupakan data yang ingin dikirimkan, dan *Signature* yang merupakan bukti kriptografi keaslian token.

Dalam penerapannya, JWT akan ditangkap oleh server dan melakukan *decoding* dari format JWT menggunakan Base64Url. Untuk keamanan, algoritma token JWT akan menggunakan *Signature* sebagai verifikator di server.

1.2. Elliptic Curve Diffie-Hellman (ECDH)

Elliptic Curve Diffie-Hellman (ECDH) merupakan protokol pertukaran kunci bersifat anonim yang memungkinkan terbentuknya rahasia bersama (*shared secret*) menggunakan pasangan kunci publik-privat kurva eliptik dari dua pihak. *Shared secret* yang terbentuk akan dihitung secara matematis oleh kedua pihak menggunakan kunci privat masing-masing dan kunci publik pihak kedua. Adapun alur algoritma ini adalah sebagai berikut:

1. Kedua pihak memilih satu persamaan kurva yang memiliki parameter yang sama. Persamaan biasanya berbentuk sebagai berikut:

$$y^2 = x^3 + ax + b$$

2. Setelahnya, baik masing-masing pihak N akan mengkalkulasi hasil perhitungan *public key* (Q) yang dimiliki dari *private key* masing-masing (d), yang kemudian akan diteruskan dalam jaringan terbuka. Adapun kalkulasi diterapkan dari rumus berikut:

$$Q_N = d_N \times G$$

3. Setelahnya, masing-masing pihak N akan mengalikan *public key* pihak lain dengan *private key* pribadi. Kedua pihak akan mendapatkan persamaan yang ekuivalen sebagai berikut:

$$S = d_A \times Q_B = d_B \times Q_A$$

Pada aplikasi E2EE modern, standar yang paling direkomendasikan adalah menggunakan kurva X25519 atau Curve25519. Dibandingkan kurva yang standar digunakan pada layanan umumnya seperti P-256, X25519 dirancang khusus untuk memiliki ketahanan bawaan terhadap *timing attacks*, yaitu serangan yang mengeksplorasi waktu pemrosesan dari sistem untuk menghasilkan suatu output, seperti algoritma enkripsi data, password, atau kunci kriptografi, dan juga menawarkan komputasi yang jauh lebih efisien, menjadikannya standar utama pada spesifikasi Web Crypto API generasi terbaru.

Proses algoritma ini memungkinkan Non-Interactive Key Exchange, mekanisme pertukaran kunci dari beberapa pihak tanpa komunikasi langsung. Ilustrasinya, jika pihak A ingin mengirim pesan kepada pihak B, A hanya perlu mengambil *Public Key* (Q) milik B yang bersifat publik, lalu mengalikannya dengan *Private Key* (d) miliknya sendiri. Secara matematis, kedua belah pihak akan menghasilkan titik *shared secret* (S) yang ekuivalen tanpa harus melakukan *handshake* secara *real-time*.

1.3. Advanced Encryption Standard (AES)

Advanced Encryption Standard (AES) adalah algoritma simetris bersifat *block cipher* yang digunakan untuk mengenkripsi dan dekripsi data yang dikirimkan. Hal ini menyebabkan AES sangat baik digunakan untuk enkripsi data seperti pesan, bukan sebagai pertukaran kunci bersama.

Adapun alur algoritma ini dijelaskan di bawah berikut:

1. Semua *byte* di dalam matriks 4x4 akan diubah dengan *byte* berbeda dari tabel [S-Box](#) (atau *Substitution Box*).

2. Setiap baris n akan digeser ke kiri sejumlah n posisi. Contoh, baris kedua akan digeser satu kali ($n = 1$).
3. Setiap kolom akan dikalikan dengan matriks 1×4 yang sudah ditentukan sebelumnya.
4. Di akhir setiap putaran, akan dilakukan operasi XOR dari matriks terakhir dengan kunci putaran.
5. Keempat proses ini akan diulang bergantung kepada jumlah bit yang digunakan (128 bit = 10 putaran, 192 bit = 12 putaran, 256 bit = 14 putaran).

Dalam perancangan sistem keamanan yang kompleks, mode operasi AES harus disesuaikan dengan kebutuhan ruang lingkup datanya:

- AES-GCM (Galois/Counter Mode): Merupakan mode *Authenticated Encryption with Associated Data* atau AEAD yang sangat cepat dan ideal untuk mengenkripsi data yang sedang tidak digunakan, seperti mengamankan *Private Key* pengguna di dalam penyimpanan lokal ataupun data pada basis data.
- AES-CTR (Counter Mode): Mengubah *block cipher* menjadi *stream cipher* dengan menggunakan *Initialization Vector* (IV) dan *counter* yang terus bertambah. Mode ini sangat ideal untuk mengenkripsi aliran pesan obrolan secara fleksibel tanpa memerlukan *padding* tambahan.

1.4. Password-based MAC Key Derivation Function (PBKDF2)

Password-based Message Authentication Code Key Derivation Function (PBKDF2) merupakan suatu *key derivation function* (KDF) yang berfungsi menerima password pengguna yang kemudian diubah menjadi kunci simetris yang kuat, kemudian digunakan untuk proses algoritma AES. Adapun proses pembuatan kunci simetris adalah sebagai berikut:

$$U_1 = \text{HMAC}(\text{Password}, \text{Salt} || i)$$

$$U_n = \text{HMAC}(\text{Password}, U_{n-1})$$

Dengan beberapa nilai U untuk setiap indeks blok i , U akan dihitung dan digabungkan dengan proses sebelumnya dengan operasi XOR untuk menciptakan hasil hash T .

$$T_i = U_1 \otimes U_2 \otimes \dots \otimes U_{n-1} \otimes U_n$$

Setelahnya, nilai T sejumlah panjang yang diinginkan L akan digabungkan menjadi sebuah *string* panjang $(T1||T2||\dots||TL)$.

1.5. Hash-based MAC Key Derivation Function (HKDF)

Hash-based Message Authentication Code Key Derivation Function (HKDF) merupakan suatu *key derivation function* (KDF) yang berfungsi menerima entropi awal (*shared secret* dari ECDH) yang memiliki fungsi serupa dengan PBKDF2. Adapun proses pembuatan kunci simetris adalah sebagai berikut:

1. Kunci akan diekstraksi lewat fungsi Hash-based MAC yang nantinya akan bersifat pseudorandom (*Pseudorandom Key*). Adapun *salt* sebagai variabel *random* dapat digunakan, namun bersifat opsional. Adapun formula yang digunakan adalah sebagai berikut:

$$PRK = HMAC(ECDH \text{ Shared Key}, Salt)$$

2. PRK (*Pseudorandom Key*) kemudian akan dipasang bersama konteks penggunaan (*Info*) dan variabel panjang yang diinginkan (L). Kemudian, dengan L sebagai batasan akan dibuat blok-blok menggunakan algoritma berikut ini:

$$U_n = HMAC (PRK, U_{n-1}, || Info || 0x0n)$$

Dengan $a||b$ berarti menggabungkan blok a dan b menjadi satu *string* yang lebih panjang.

Proses dilakukan sampai T terakhir, dimana semua blok akan digabungkan $(T1||T2||\dots||TL)$ yang akan menjadi kunci simetris yang digunakan untuk AES.

1.6. Hashing SHA-256

Hash digunakan untuk menerima input dalam bentuk dan ukuran apapun untuk menghasilkan keluaran string unik yang selalu berukuran sama. Dalam praktiknya, setiap masukan yang sama akan mengeluarkan keluaran yang sama, dan sangat tidak mungkin untuk menemukan dua masukan yang mengeluarkan keluaran yang sama. Adapun algoritma hashing yang digunakan adalah SHA-256, yang beralur seperti berikut:

1. Setiap masukan akan ditambah 1 bit di akhir masukan, kemudian akan ditambah *padding* bit 0 sampai panjang masukan + *padding* berjarak 64 bit dari kelipatan 512 selanjutnya. Di dalam 64 bit tersebut, akan dimasukkan panjang masukan awal.
2. Secara *default*, SHA-256 akan menginisialisasi 8 variabel 32 bit ($H_0, H_1, \dots, H_7$) yang diturunkan dari hasil akar delapan bilangan prima.
3. Kompresi kemudian dilakukan dengan memisahkan setiap 512 bit menjadi 64 buah 32 bit *string*, yang kemudian akan di-loop sebanyak 64 kali dengan melakukan operasi bitwise *right shift*, kemudian mengembalikan LSB menjadi MSB, dan gerbang logika sederhana XOR, AND, dan NOT.
4. Setelah semua blok diproses ($H_0 - H_7$), kedelapan blok ini akan digabung menjadi *string* 64 bit sebagai hasil *hashing*.

1.7. Message Authentication Code (MAC)

Message Authentication Code (MAC) adalah fungsi kriptografi yang menjamin integritas pesan, memastikan pesan tidak diubah, dan autentikasi, yaitu memastikan pesan benar-benar berasal dari pihak yang memiliki kunci secara sah. Algoritma yang umum digunakan adalah HMAC atau *Hash-based MAC* yang dipadukan dengan SHA-256. Dalam arsitektur kriptografi klasik yang kokoh, paradigma yang standar digunakan adalah Encrypt-then-MAC. Paradigma ini mengenkripsi *plaintext* terlebih dahulu menjadi *ciphertext*, kemudian menghitung nilai MAC dari *ciphertext* tersebut sekaligus dengan metadatanya. Selain itu,

paradigma ini juga secara matematis terbukti mampu mencegah berbagai kelas serangan aktif, termasuk *Padding Oracle Attack* dan *Replay Attack*.

1.8. Dockerization

Dalam pengerjaan spek bonus, digunakan Docker sebagai kontainerisasi, Nginx sebagai *reverse proxy*, Caddy sebagai penyedia layanan sertifikasi SSL, dan nip.io sebagai penyedia layanan DNS.

BAB II

PERANCANGAN DAN IMPLEMENTASI

2.1. Abstraksi dan Arsitektur Sistem

Sistem dirancang menggunakan arsitektur *client-server* sederhana yang mengimplementasikan autentikasi, pembentukan kunci dan penurunannya, dan enkripsi pesan. *Server* disini bertugas sebagai perantara “buta” dimana semua data yang disimpan sudah terenkripsi dari pihak klien. Operasi kriptografi akan menggunakan Web Crypto API, sebagaimana dianjurkan oleh dokumen penugasan.

2.2. Registrasi dan Manajemen Pengguna

Sistem akan menerima masukan email dan password dari pengguna, menimbang:

1. Tidak akan ada pengecekan keaslian email karena bukan di dalam cakupan penugasan ini.
2. Klien secara lokal akan membuat pasangan kunci asimetris untuk algoritma ECDH.
3. Password klien akan di-hash menggunakan algoritma SHA-256 dan dilakukan *salting* (dalam hal ini akan menggunakan string *random* 16 bit).
4. Detail yang akan disimpan di database adalah email, password yang sudah di-hash, salt, *public key*, dan *private key*.

Dalam implementasinya, keempat aspek ini diterapkan setidaknya pada file `auth.ts` baik di folder frontend dan backend, serta konfigurasi supabase di `db/supabase.ts`.

2.3. Autentikasi Pengguna (JWT)

Secara garis besar, autentikasi pengguna akan mencocokkan hasil *hashing* masukan password dengan *salt* terkait dengan password yang disimpan pada database. Adapun alur autentikasi adalah sebagai berikut:

1. Jika verifikasi valid, sistem akan memberikan JWT sebagai token akses kepada pengguna.
2. JWT dalam format JWS akan ditandatangani secara digital menggunakan Elliptic Curve Digital Signature Algorithm (ECDSA) dengan 3 varian , yaitu ES256(P-256), ES384(P-384). Dan ES512(P-512).
3. Adapun modul yang diminta (sign dan verify) untuk operasi JWT diimplementasikan pada file `jwt/index.ts` dan `jwt/ecdsa.ts`.

2.4 Pembentukan Kunci Komunikasi (Key Exchange)

Proses pembentukan terjadi dengan menggunakan protokol pertukaran kunci asimetris lalu diturunkan menjadi kunci simetris. Adapun urutan protokol sebagai berikut:

1. Elliptic Curve Diffie Hellman

Saat user memilih salah satu kontak untuk berkomunikasi. Pada fungsi `deriveSessionKeys`, sistem akan langsung mengambil *public key* terpilih dari server lalu menjalankan algoritma ECDH. Implementasi hal ini menggunakan algoritma **X25519**. Dengan menggabungkan *private key* user dengan *public key* terpilih, akan menghasilkan *shared secret* berukuran 256 bit pada keduanya

2. HMAC based Extract and Expand Key Derivation Function

Setelah dihasilkan *shared secret*, sistem menerapkan HKDF dengan *hash* SHA 256. Ini bertujuan agar *shared secret* yang dihasilkan tidak langsung digunakan sebagai kunci enkripsi. Demi memastikan keunikan kunci, sistem membuat *salt* dengan menggabungkan email sesuai dengan urutan alfabet (`email_1|email_2`). String yang digunakan sebagai parameter adalah `ii4021-crypto-chat-v1`(di encode ke UTF-8)

3. Penurunan Kunci Sesi

Selanjutnya, HKDF memperbesar *shared secret* menjadi 512 bit yang dibagi menjadi dua bagian

- AES sebagai kunci simetris AES-256-CTR untuk mengamankan isi pesan

- HMAC untuk HMAC-SHA256 imtil memvalidasi isi pesan

4. Penurunan Kunci Sesi

Sistem tidak mengirimkan *private key* ke *server* dalam bentuk *plaintext*.

Sebaliknya, sistem mengimplementasikan Key Wrapping:

1. Saat registrasi, klien membangkitkan pasangan kunci asimetris X25519.
2. *Password* pengguna diubah menjadi kunci simetris kuat menggunakan PBKDF2 (dengan parameter keamanan tinggi: 100.000 iterasi dan *salt* acak).
3. *Private Key* kemudian dibungkus (dienkripsi) menggunakan AES-256-GCM dengan kunci turunan PBKDF2 tersebut, lalu dikirim ke *database* bersama dengan parameter IV dan *salt*.

Saat registrasi, aplikasi secara sadar menggunakan metode peng-*export*-an kunci (*extractable: true*) sesaat untuk dienkripsi oleh AES-GCM sebelum dihapus oleh *Garbage Collector* memori. Celah yang dapat dieksploitasi dari pengeksposan *raw bytes* di dalam *heap memory* dari JavaScript ini telah dianalisis dan dinyatakan aman karena kerentanan hanya berlangsung selama beberapa milidetik. Pada saat sesi aktif dan melakukan obrolan dengan pihak lain, *Private Key* yang diimpor ulang bersifat *extractable: false*, sehingga aplikasi sepenuhnya kebal terhadap upaya pencurian kunci yang dilakukan melalui vektor serangan *Cross-Site Scripting* (XSS)

2.5 Enkripsi dan Dekripsi Pesan

Penerapan enkripsi pesan dilakukan ketika klien ingin mengirimkan pesan ke server menggunakan algoritma simetris AES-256, dimana variasi yang diterapkan adalah AES-256-CTR untuk merahasiakan pesan dan validasi keutuhan selama transit ke server.

Server hanya menerima hasil enkripsi pesan, dan tidak memiliki kunci secara eksplisit, karena kunci hanya dipegang dari *key exchange*. Ketika terdapat salah satu pihak yang keluar dari percakapan atau satu pihak tidak dapat bercakap, maka proses percakapan tidak akan didekripsi.

Adapun penerapan implementasi ini terdapat pada file `messageCrypto.ts`, `MessageComposer.tsx`, dan `messages.ts`.

2.6 Implementasi Bonus

1. Integritas dan Autentikasi Pesan (MAC)

Sistem menghitung MAC sebelum pengiriman pesan dilakukan untuk mencegah modifikasi data. Ketika pesan sampai ke tujuan, maka nilai MAC akan dihitung kembali menggunakan HMAC-SHA256. Implementasi dapat ditemukan di file `messageCrypto.ts`.

2. Unit Test

Library `sign` dan `verify` milik JWT sudah dilengkapi dengan unit test pada file `jwt.test.ts`. File ini bertujuan untuk menguji kegagalan validasi, header yang tidak valid, algoritma yang berbeda, dan mitigasi parameter yang kadaluarsa.

3. Docker dan Deployment

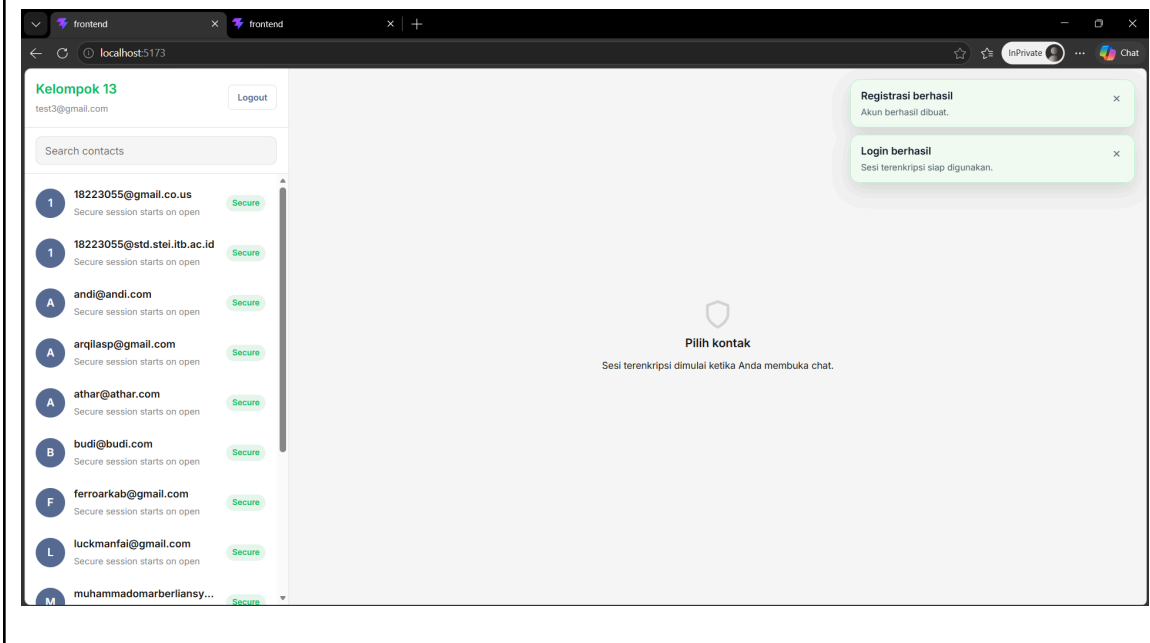
Docker diimplementasikan bersama konfigurasi `nginx` untuk memudahkan setting *proxy* kepada HTTPS. Hal ini dibutuhkan karena Web Crypto API hanya dapat berjalan pada lingkungan yang dianggapnya aman, maka digunakannya layanan DNS nip.io untuk mempermudah *deployment*. Adapun *reverse proxy* juga didukung oleh Caddy yang memberikan sertifikasi SSL dari Let's Encrypt. Implementasi dapat dilihat di setiap `Dockerfile`, `docker-compose.yml`, `nginx.conf`, dan `Caddyfile`.

BAB III

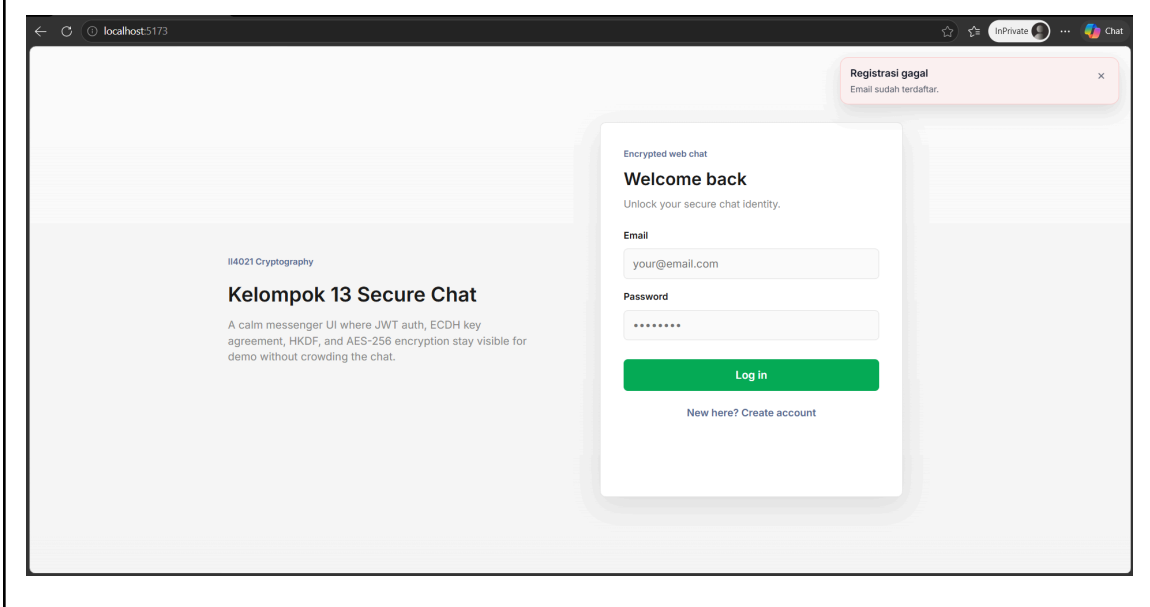
PENGUJIAN DAN HASIL

3.1. Uji Autentikasi dan Manajemen Pengguna

Registrasi dengan data valid (Berhasil)



Registrasi dengan email yang sudah pernah dipakai (Gagal)



Registrasi dengan password yang tidak sama atau kurang dari 8 karakter (Gagal)

The image displays two screenshots of a web application interface for creating a secure account. The application is running on localhost:5173.

Top Screenshot: The 'Create secure account' form is shown. The 'Email' field contains 'test4@gmail.com'. The 'Password' field contains 7 characters (indicated by dots). The 'Confirm Password' field contains 7 characters (indicated by dots). A red error message 'Konfirmasi password tidak sama' is displayed below the 'Confirm Password' field. A green 'Create account' button is visible. A yellow error message 'Form registrasi belum valid' is displayed in the top right corner, stating: 'Pastikan email valid, password >= 8 karakter, dan konfirmasi password sama.'

Bottom Screenshot: The 'Create secure account' form is shown again. The 'Email' field contains 'test4@gmail.com'. The 'Password' field contains 8 characters (indicated by dots). The 'Confirm Password' field contains 8 characters (indicated by dots). A red error message 'Minimal 8 karakter' is displayed below the 'Password' field. A green 'Create account' button is visible. A yellow error message 'Form registrasi belum valid' is displayed in the top right corner, stating: 'Pastikan email valid, password >= 8 karakter, dan konfirmasi password sama.'

Registrasi dengan password yang tidak sama atau kurang dari 8 karakter (Gagal)

The image displays two screenshots of a web application interface for creating a secure account. The interface is titled "Kelompok 13 Secure Chat" and includes a description: "A calm messenger UI where JWT auth, ECDH key agreement, HKDF, and AES-256 encryption stay visible for demo without crowding the chat." The registration form is titled "Create secure account" and includes fields for "Email" (test4@gmail.com), "Password", and "Confirm Password". The "Password" field is highlighted with a red border, and the "Confirm Password" field is also highlighted with a red border. A yellow error message is displayed in the top right corner, stating: "Form registrasi belum valid. Pastikan email valid, password >= 8 karakter, dan konfirmasi password sama." The "Create account" button is green, and there is a link for "Already have an account? Log in".

Client-side keys

Create secure account

Your chat keys are prepared in this browser.

Email

Password

Minimal 8 karakter

Confirm Password

Konfirmasi password tidak sama

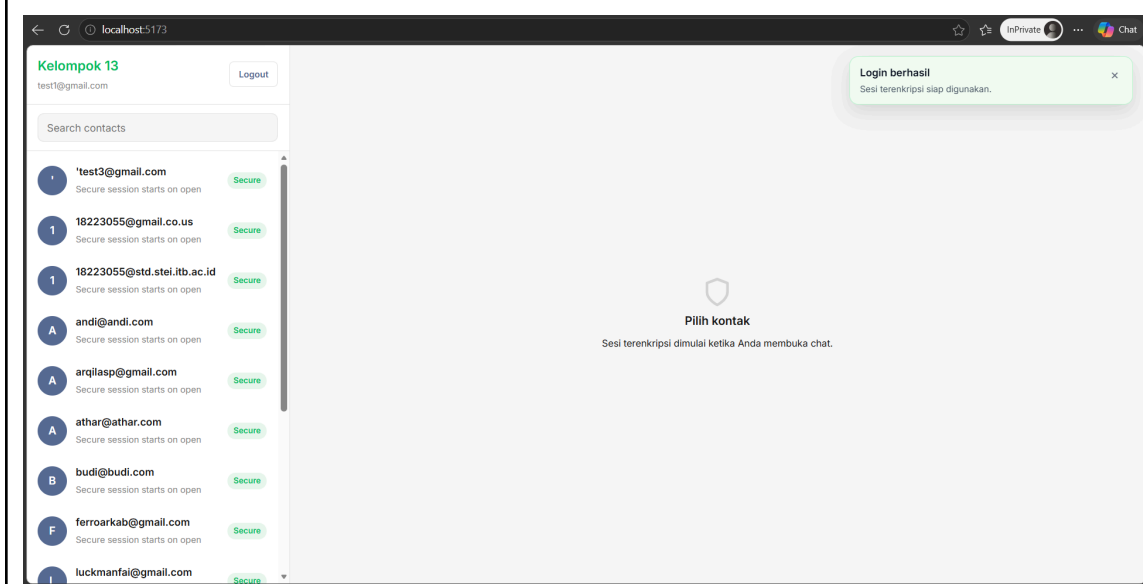
Create account

Already have an account? Log in

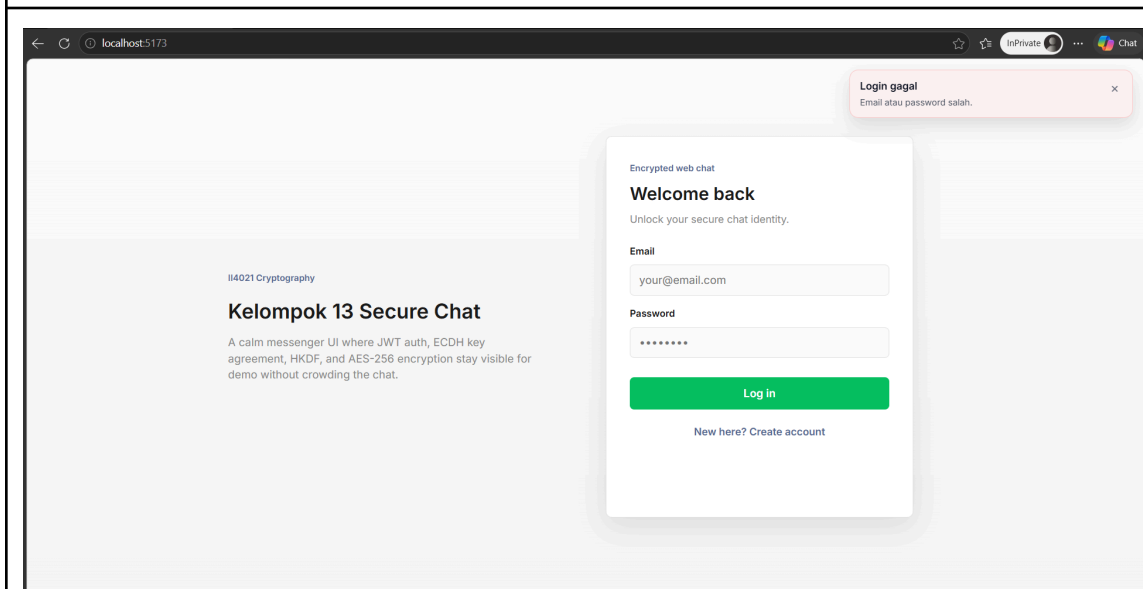
Form registrasi belum valid

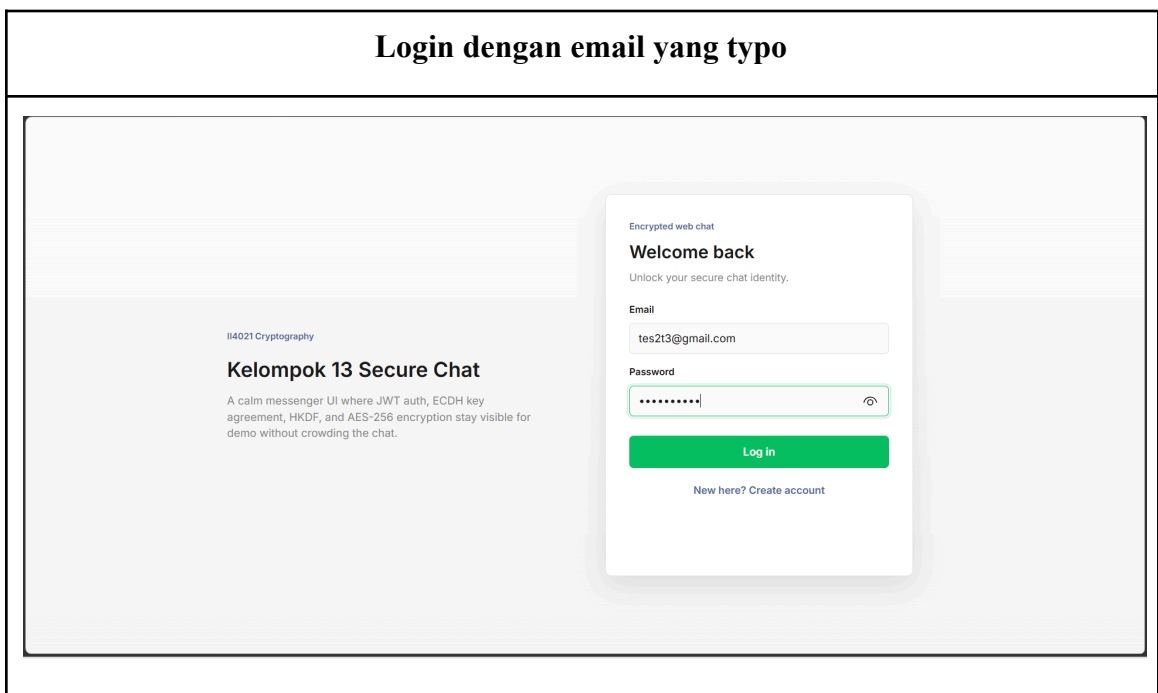
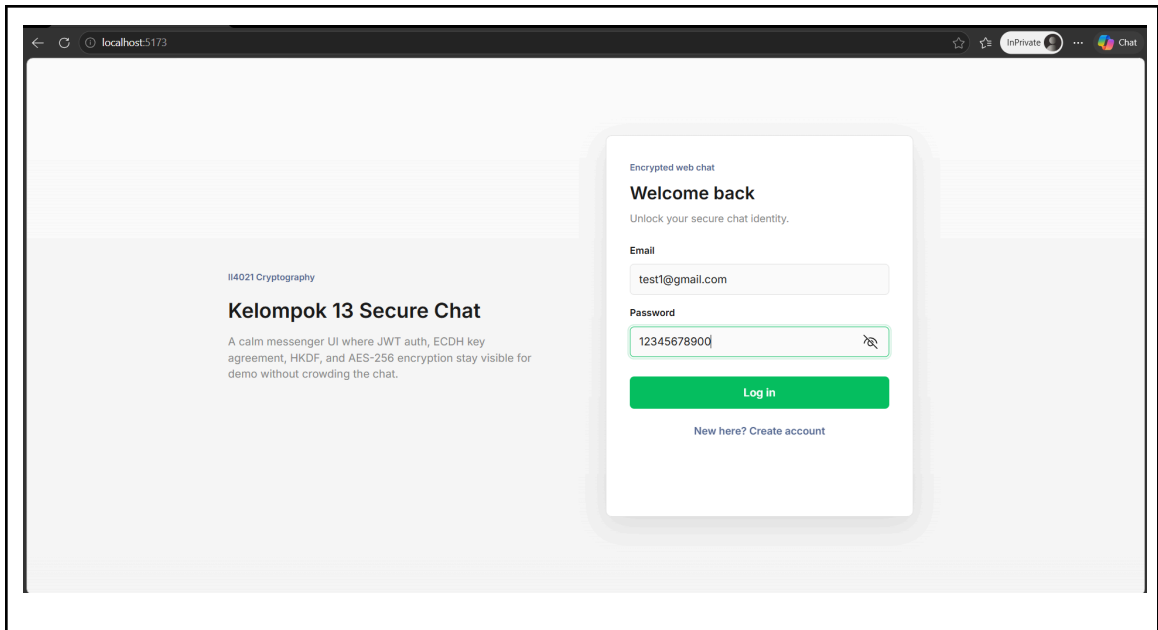
Pastikan email valid, password >= 8 karakter, dan konfirmasi password sama.

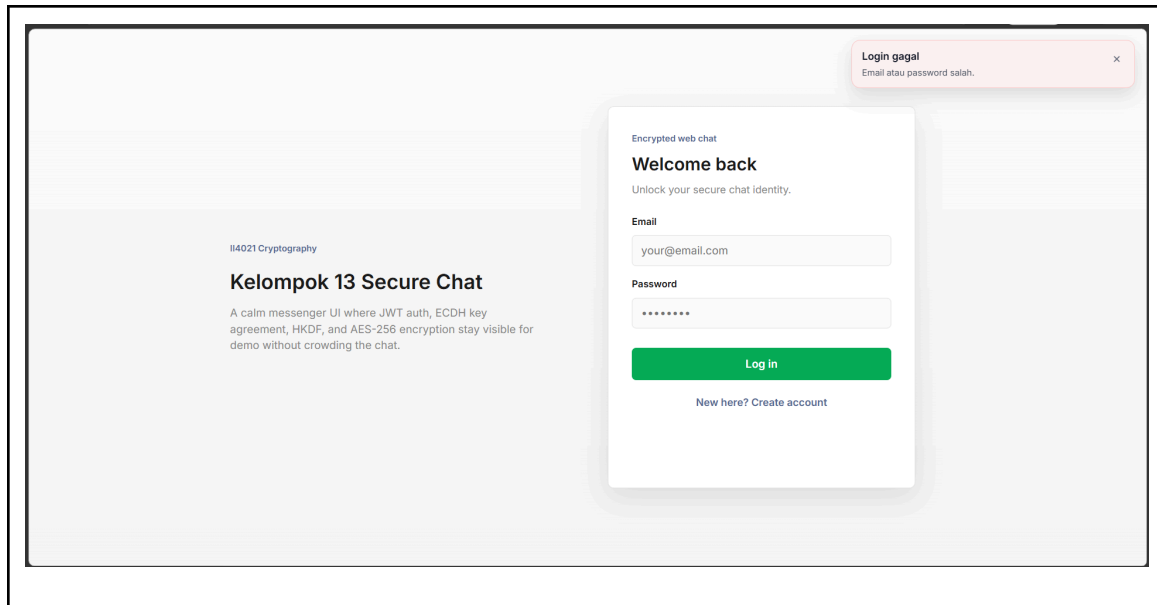
Login dengan password benar



Login dengan password salah







3.2. Unit Testing

- Frontend (Uji pembentukan kunci komunikasi (ECDH dan KDF), enkripsi dan dekripsi pesan, dan integritas dan autentikasi pesan (MAC))

Aspek	Tercapai	Log
Shared secret identik	Ya	✓ ECDH Key Exchange & KDF > Alice dan Bob menghasilkan shared secret yang identik dari sisi masing-masing
Derivasi kunci (AES & HMAC)	Ya	✓ ECDH Key Exchange & KDF > shared secret berhasil diturunkan menjadi kunci AES-256-CTR dan HMAC-SHA256
Dekripsi silang	Ya	✓ ECDH Key Exchange & KDF > session keys yang identik memungkinkan komunikasi dua arah
Dekripsi berhasil	Ya	✓ Enkripsi dan Dekripsi Pesan > pesan berhasil didekripsi menjadi plaintext dengan kunci yang benar
Dekripsi gagal (kunci salah)	Ya	✓ Enkripsi dan Dekripsi Pesan > dekripsi gagal jika menggunakan kunci yang salah (MiTM tidak dapat membaca)
Pembuatan IV unik	Ya	✓ Enkripsi dan Dekripsi Pesan > setiap pesan menghasilkan IV yang unik (mencegah nonce reuse)
Penerimaan MAC valid	Ya	✓ Integritas & Autentikasi Pesan (MAC) > pesan dengan MAC valid diterima dan didekripsi dengan

		benar
Penolakan modifikasi ciphertext	Ya	✓ Integritas & Autentikasi Pesan (MAC) > modifikasi ciphertext memicu kegagalan MAC sebelum dekripsi dilakukan
Penolakan modifikasi MAC	Ya	✓ Integritas & Autentikasi Pesan (MAC) > modifikasi field MAC secara langsung ditolak
Penolakan penukaran Pengirim/Penerima	Ya	✓ Integritas & Autentikasi Pesan (MAC) > MAC mengikat konteks pengirim/penerima – swap sender-receiver ditolak
Penolakan pemalsuan timestamp	Ya	✓ Integritas & Autentikasi Pesan (MAC) > MAC mengikat timestamp - pemalsuan waktu kirim pesan ditolak
Validasi timestamp ekuivalen	Ya	✓ Integritas & Autentikasi Pesan (MAC) > timestamp ekuivalen dengan format offset tetap lolos validasi MAC

- Backend (Uji implementasi JSON Web Token (JWT))

Aspek	Tercapai	Log
Struktur tiga bagian	Ya	✓ sign – happy path > menghasilkan string tiga bagian yang dipisahkan titik
Encoding algoritma di header	Ya	✓ sign – happy path > menyandikan algoritma yang benar pada header
Prioritas klaim	Ya	✓ sign – happy path > menggabungkan payload dan claims; claims diprioritaskan dibanding key payload
Dukungan signing ES384	Ya	✓ sign – happy path > berfungsi dengan ES384
Dukungan signing ES512	Ya	✓ sign – happy path > berfungsi dengan ES512
Dukungan klaim yang dihilangkan	Ya	✓ sign – happy path > berfungsi saat semua claims dihilangkan

Penanganan algoritma tidak valid	Ya	✓ sign – edge cases > melempar error saat alg bukan ES256 ES384 ES512
Penanganan payload tak terserialisasi	Ya	✓ sign – edge cases > melempar error saat payload tidak bisa diserialisasi menjadi JSON
Penanganan ketidakcocokan kurva kunci	Ya	✓ sign – edge cases > melempar error atau menghasilkan token tak terverifikasi saat kurva key tidak cocok dengan algoritma
Penerimaan exp kedaluwarsa	Ya	✓ sign – edge cases > menerima exp yang sudah kedaluwarsa tanpa melempar error
Penanganan private key kosong	Ya	✓ sign – edge cases > melempar error saat private key berupa string kosong
Pelestarian tipe payload	Ya	✓ sign – edge cases > mempertahankan nilai numerik dan boolean pada payload
Decoding token valid	Ya	✓ verify – happy path > mengembalikan header, payload, dan signature terdecode untuk token valid
Validasi klaim standar	Ya	✓ verify – happy path > memvalidasi iss, sub, aud, dan jti saat ditentukan di opsi
Opsi abaikan exp	Ya	✓ verify – happy path > mengabaikan exp saat ignoreExp bernilai true
Opsi abaikan nbf	Ya	✓ verify – happy path > mengabaikan nbf saat ignoreNbf bernilai true
Verifikasi ES384	Ya	✓ verify – happy path > menerima token yang ditandatangani dengan ES384
Verifikasi ES512	Ya	✓ verify – happy path > menerima token yang ditandatangani dengan ES512 [16.00ms]
Penanganan token tidak lengkap	Ya	✓ verify – edge cases > melempar error saat token memiliki kurang dari tiga bagian
Penanganan manipulasi signature	Ya	✓ verify – edge cases > melempar error saat signature dimanipulasi

Penanganan public key salah	Ya	✓ verify – edge cases > throws when verified with the wrong public key
Penanganan token kedaluwarsa	Ya	✓ verify – edge cases > throws when token is expired
Penanganan nbf di masa depan	Ya	✓ verify – edge cases > throws when nbf is in the future
Penanganan algoritma tak diizinkan	Ya	✓ verify – edge cases > throws when alg is not in the allowed list
Penanganan ketidakcocokan iss	Ya	✓ verify – edge cases > throws when iss does not match
Penanganan ketidakcocokan sub	Ya	✓ verify – edge cases > throws when sub does not match
Penanganan ketidakcocokan aud	Ya	✓ verify – edge cases > throws when aud does not match
Validasi array aud	Ya	✓ verify – edge cases > accepts aud claim as an array containing the expected audience
Penanganan ketidakcocokan array aud	Ya	✓ verify – edge cases > throws when aud is an array that does not contain the expected audience
Penanganan ketidakcocokan jti	Ya	✓ verify – edge cases > throws when jti does not match
Penanganan JSON header tidak valid	Ya	✓ verify – edge cases > throws when the header segment is not valid JSON
Penanganan JSON payload tidak valid	Ya	✓ verify – edge cases > throws when the payload segment is not valid JSON
Penanganan segmen kosong	Ya	✓ verify – edge cases > throws when any segment is empty

BAB IV

KESIMPULAN

Berdasarkan hasil implementasi dan pengujian, aplikasi secure chat berbasis web ini telah berhasil memenuhi spesifikasi utama tugas. Sistem mendukung registrasi dan login pengguna, autentikasi menggunakan JWT, daftar kontak, pembentukan kunci komunikasi dengan ECDH, penurunan kunci menggunakan HKDF, serta enkripsi dan dekripsi pesan menggunakan AES-256. Seluruh operasi kriptografi utama dilakukan di sisi klien, sementara server hanya menyimpan dan meneruskan data terenkripsi tanpa mengetahui isi plainteks pesan. Sistem juga mengimplementasikan tiga fitur bonus, yaitu integritas dan autentikasi pesan menggunakan MAC, unit test untuk library JWT, serta deployment menggunakan Docker. MAC digunakan untuk memastikan pesan tidak dimodifikasi sebelum didekripsi, unit test JWT memastikan fungsi sign dan verify berjalan benar pada skenario normal maupun edge case, sedangkan Docker mempermudah proses menjalankan aplikasi secara konsisten di berbagai lingkungan. Pengujian pada sistem mencakup registrasi, login, JWT, ECDH-HKDF, enkripsi-dekripsi AES-256, dan validasi MAC, hasilnya sistem berjalan sesuai spesifikasi. Penerbitan JWT, dekripsi pesan dengan kunci yang benar, input tidak valid, token salah, kunci keliru, atau pesan yang dimodifikasi berhasil ditolak atau ditandai tidak valid

DAFTAR PUSTAKA

- GeeksforGeeks. "Advanced Encryption Standard (AES)." GeeksforGeeks, 15 Oct. 2021, www.geeksforgeeks.org/computer-networks/advanced-encryption-standard-aes/.
- Jones, M., Bradley, J., & Sakimura, N. (2015). *RFC 7519: JSON Web Token (JWT)*. Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc7519>
- Jones, M., Bradley, J., & Sakimura, N. (2015). *RFC 7515: JSON Web Signature (JWS)*. Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc7515>
- Langley, A., Hamburg, M., & Turner, S. (2016). *RFC 7748: Elliptic Curves for Security (Curve25519/X25519)*. Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc7748>
- Mozilla Developer Network (MDN). (2024). *SubtleCrypto Interface - Web APIs*. MDN Web Docs. Tersedia di: <https://developer.mozilla.org/en-US/docs/Web/API/SubtleCrypto>
- Munir, Rinaldi. Beberapa Fungsi Hash (MD5, SHA-1, SHA-256, SHA-3). STEI ITB, 2026.
- . Elgamal Signature Scheme, DSA, ECDSA, Dan Schnorr Signature Scheme. STEI ITB, 2026.

LAMPIRAN

i . Link Repository

[OmarBerliansyah/ii4021-tugas-3-kelompok-13](https://github.com/OmarBerliansyah/ii4021-tugas-3-kelompok-13)

<https://20.2.249.137.nip.io/>

II. Link Video Demo

https://youtu.be/P9GBdAaR_gI

III. Pembagian Tugas

Nama	NIM	Pembagian Tugas
Ferro Arka Berlian	18223027	Backend (Autentikasi dan Chat), Finalisasi Frontend, Penyusunan Laporan
Muhammad Omar Berliansyah	18223055	Integrasi Backend dan Frontend, Bonus MAC dan Unit Test, Penyusunan Laporan
Atharizza Muhammad Athaya	18223079	Frontend (Autentikasi dan Chat), Bonus Docker, Penyusunan Laporan